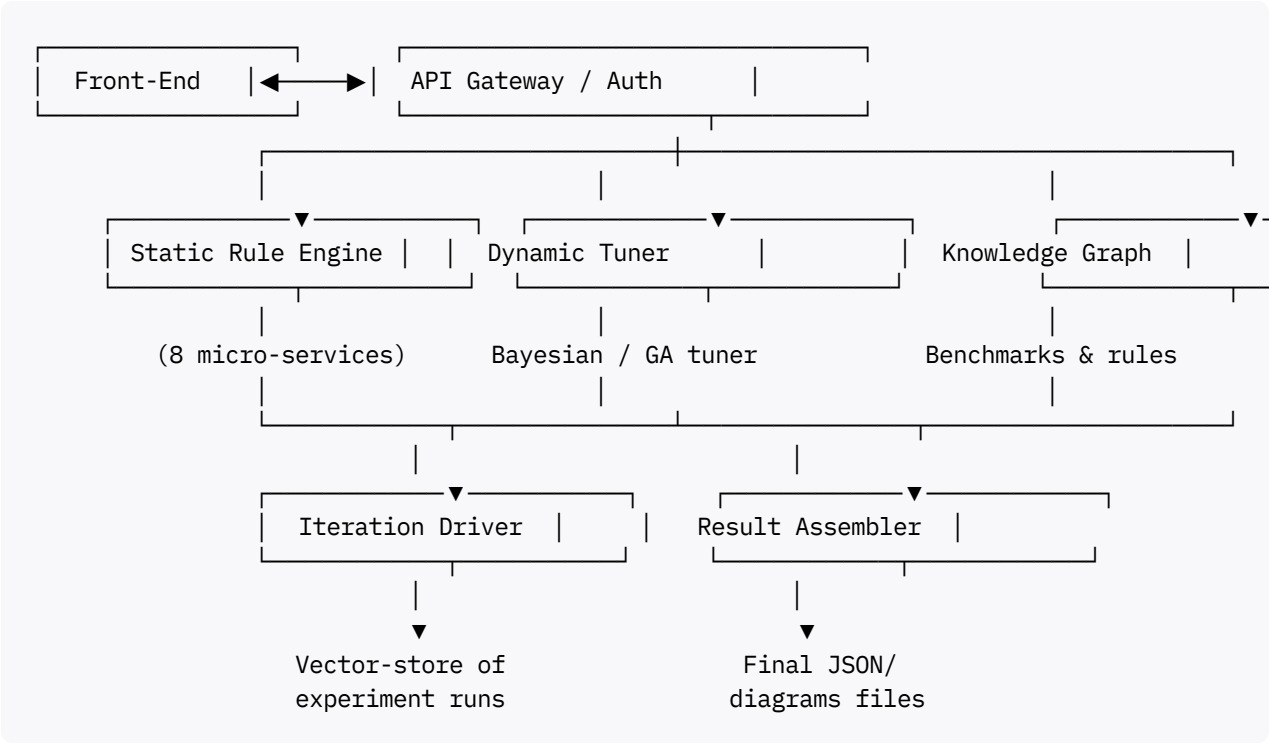
Evaluation-Driven RAG Design Blueprint

Below is a complete design package—diagrams, APIs, algorithms, and evaluation logic—for a **RAG Evaluation Module** that helps teams converge on an optimal ingestion & retrieval stack through **static (rule-based)** and **dynamic (auto-tuned)** workflows.

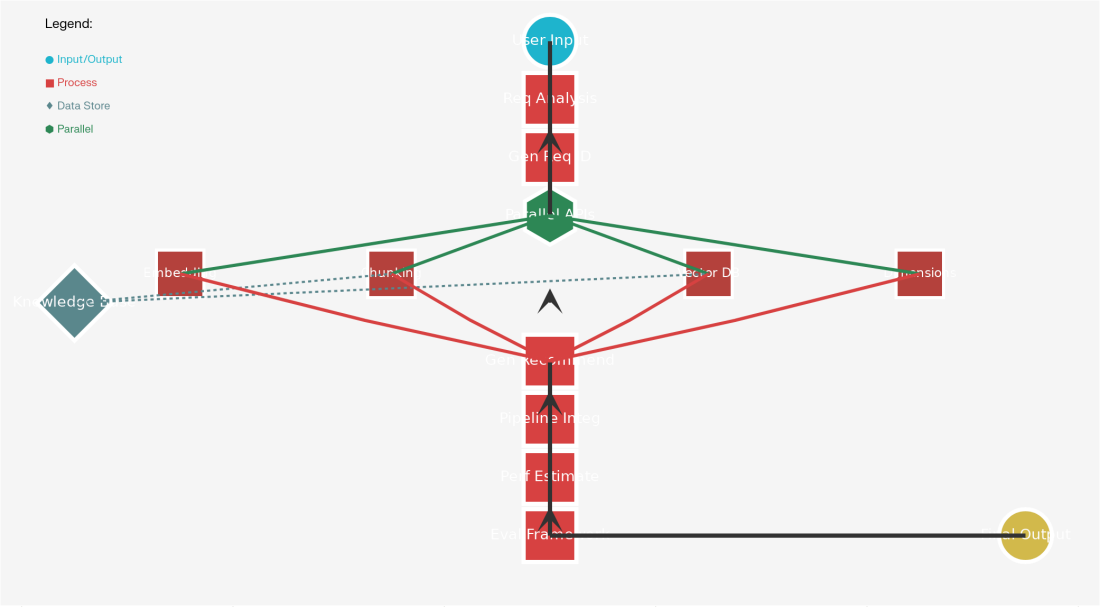
1 | High-Level Architecture

The module is delivered as nine REST services behind a single gateway.



2 | Data-Flow & Iteration Diagram

RAG Pipeline Flow Diagram



RAG Pipeline Recommendation System Data Flow

1. **Collect Inputs** → persist `requirements_id`.
2. **Static Pass** → rule engine emits seed configs for all eight axes.
3. **Dynamic Loop (≤N)**
   a. Generate candidate configs with Bayesian Opt./Genetic Alg.
   b. Run micro-benchmarks (embedding recall, chunking coverage, etc.).
   c. Persist metrics, update posterior.
   d. Stop when Δ-score < ε or max_iter.
4. **Assembler** picks top-K and builds end-to-end pipeline spec.

3 | User-Input Contract (`POST /user-requirements`)

Field	Type	Why it matters
`data_sources[]`	list[{`type`, `format`, `count`, `avg_size_kb`}]	Drives chunking & ingestion throughput
`sample_payloads`	array[blob/URI]	Used for on-the-fly micro-tests
`use_case`	enum(`kb_qa`,`agent`,`summarizer`,…)	Sets accuracy/latency targets
`growth_forecast`	{now, 12 mo}	Guides index & DB scaling
`priority`	ordered list (`accuracy`, `latency`, `cost`)	Weight vector for scoring
`privacy_level`	enum(`public`,`pii`,`highly_sensitive`)	Filters cloud vs on-prem DBs

Field	Type	Why it matters
`query_complexity`	enum(`keyword`,`semantic`,`reasoning`)	Affects embedding & LLM choice
`latency_budget_ms`	int	Retriever & chunk size tuning
`accuracy_min`	float(0-1)	Early-stop threshold
`truthfulness_tolerance`	float(0-1)	Chooses reranking / verification
`budget_cap_usd`	optional	Cost-aware pruning

4 | Eight Evaluation APIs

4.1 Embedding Model (`POST /eval/embedding`)

- **Static**: rule table keyed on language, domain, GPU access.
- **Dynamic**: run MTEB sub-tasks on sample → compute `mean_recall`.
- **Score** = $w_1 \cdot \text{recall} + w_2 \cdot \text{speed} + w_3 \cdot \$$
- **Output**: `{model_id, dims, batch_size, fp16, recall@10, tok_cost}`

4.2 Chunking Strategy (`POST /eval/chunking`)

- Static heuristics (PDF → hierarchical, code → AST, etc.).
- Dynamic grid search on `(size, overlap)` → minimize `Δ-BLEU` between original doc and reconstruction.
- Metrics: coverage%, redundancy%, reconstruction-BLEU.

4.3 Dimension Reduction (`POST /eval/dimensions`)

- Static: default dims by use case (search 768, sim 384, class 1,024).
- Dynamic: PCA/Autoencoder sweeps; stop when `recall_drop < 2 pp`.

4.4 LLM Selection (`POST /eval/llm`)

- Static: mapping table (reasoning → GPT-4o, basic → Gemma-7B).
- Dynamic: small subset of queries run; score with ROUGE-L, ROGUE, Faithfulness.

4.5 Index Type (`POST /eval/index`)

- Static:
  - ≤1 M vec → IVF-SQ
  - 1-50 M → HNSW
  - 50 M → DiskANN or PQ-HNSW

- Dynamic: measure build-time vs recall@10 on sample.

4.6 Vector DB (`POST /eval/vectordb`)

Rule filters (cloud, PII) then dynamic latency tests using Docker playgrounds of Qdrant, Milvus, Redis, MongoDB Atlas VSearch.

4.7 Retrieval Config (`POST /eval/retriever`)

Search top-k, `ef_search`, hybrid weights. Dynamic hyper-search maximizes F1 between ground-truth passages and retrieved set.

4.8 Prompt Template (`POST /eval/prompt`)

Template library tagged by task. Dynamic evolutionary search varying system/instruction/format context; evaluated with GPTJudge for relevance & faithfulness.

5 | Aggregator API (`POST /recommend/pipeline`)

Returns:

```
{
  "pipeline": {
    "ingestion": {...},
    "retrieval": {...},
    "generation": {...}
  },
  "rationale": "...weighted score details...",
  "expected_metrics": {
    "precision@5": 0.82,
    "latency_p95_ms": 180,
    "monthly_cost_usd": 235
  },
  "iteration_logs_uri": "s3://runs/req-123/*.json"
}
```

6 | Low-Level Design Highlights

Component	Key Classes	Core Algorithms
`RuleEngine`	`RuleSet`, `Evaluator`	YAML→Decision-Tree
`DynamicTuner`	`SearchSpace`, `BayesOpt`, `GA`	Optuna or Ax
`MetricRunner`	`EmbedBench`, `ChunkBench`, `LLMBench`	micro-bench executors
`ResultStore`	PostgreSQL + Parquet	stores each trial
`Scorer`	weighted linear / Pareto front	multi-objective

Evaluation math example (Embedding):

$$\text{score} = w_{acc} \cdot R@10 + w_{lat} \cdot \frac{1}{\text{ms}} + w_{cost} \cdot \frac{1}{\$}$$

Weights come from `priority` list (softmax-normalized).

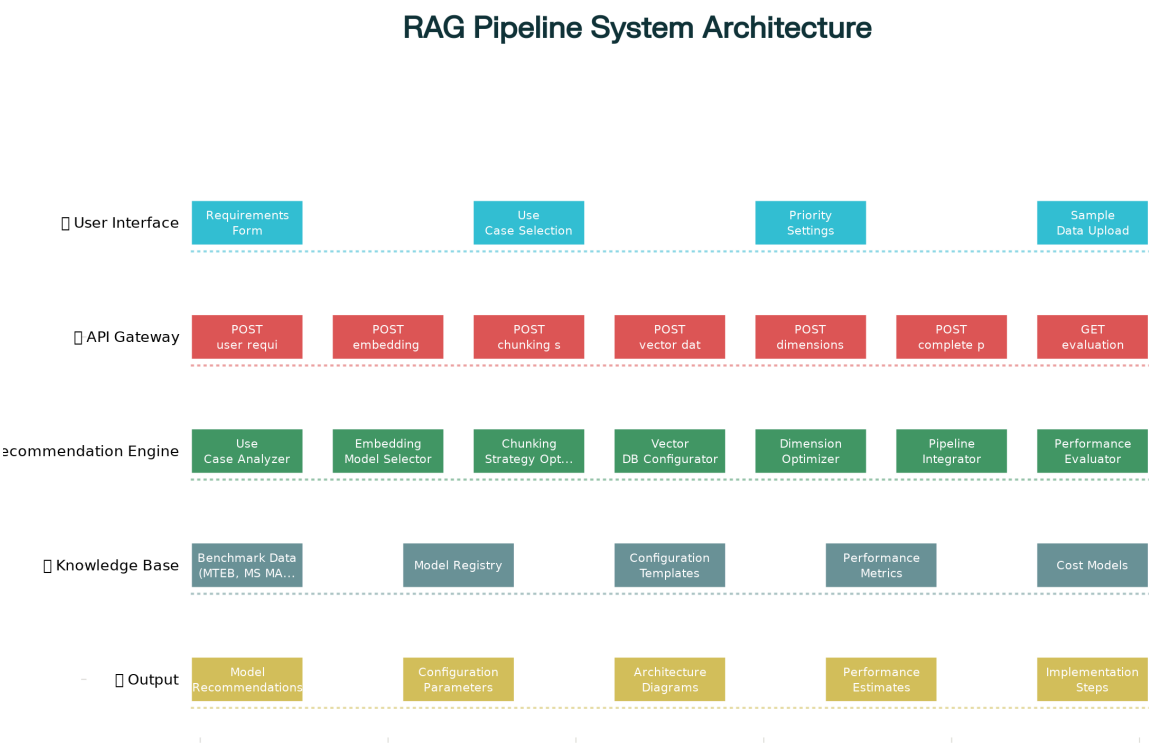
Stopping rule:

```
if iter ≥3 and (score_best - score_last) < ε: break
```

Default `ε = 0.005`.

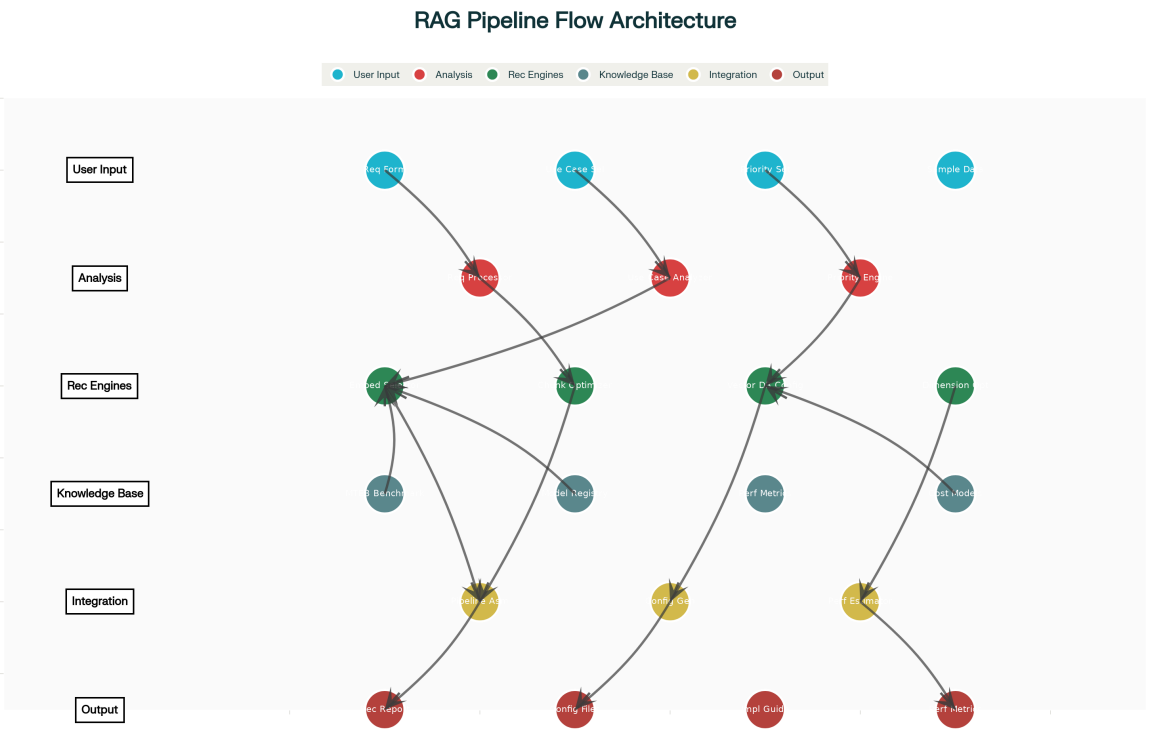
7 | Output Artifacts

1. **Architecture diagram** – system layers & data stores



RAG Pipeline Recommendation System Architecture

2. **Data-flow diagram** – iteration loop & micro-services
3. **Component interaction diagram** – dependencies & contracts



RAG Pipeline Components Interaction Diagram

4. **Markdown spec** – full API schemas & example payloads

Each diagram is downloadable and ready for inclusion in design documents.

This blueprint gives engineers everything required—APIs, iteration algorithms, evaluation metrics, and design visuals—to implement an evaluation-first RAG recommender that converges on the best pipeline for any dataset and priority set.